



Common Problems

Notification System ·||·

Multi-step Processes

Scaling Writes

By Evan King · Published Jul 28, 2026 · medium

Understanding the Problem

What is a Notification System?

A notification system is an internal platform that other teams at a company use to send messages to users across channels like push, email, and SMS. Product services hand it a message and a recipient, and the platform takes care of actually delivering it.

This is a platform problem rather than a user-facing product, so our customers are other engineering teams inside the company. Notification systems are used to send messages to users, so internal clients are like the auth service that sends one-time passwords or the marketing team that blasts a promo to a million users at once.

Functional Requirements

This is the set I steer candidates toward when I ask this question. There are a lot of different ways this system can go so interviewers will try to get you to cover some of the key ones while they let you get creative with the rest.

Core Requirements

1. Upstream services should be able to send a notification to a user via push, email, or SMS, either immediately or in the future.
2. Upstream services should be able to send campaigns, the same message delivered to a whole segment of users, immediately or scheduled.
3. Users should be able to set notification preferences (channel opt-outs and quiet hours).

Below the line (out of scope):

- Template management and rich content authoring.
- Delivery analytics dashboards (open rates, click-through tracking).

- In-app notification feeds and badge counts.
- Frequency capping across notification types.

Remember that you're not getting much credit (if any) for enumerating "below the line" requirements, so skip these if you're low on time.

Non-Functional Requirements

Before writing these down, we need a number to design against. This is a platform question, so how the system holds up under load is most of what we are being asked here.

Say we deliver 10M notifications a day. Spread evenly, that's about 100 per second which is honestly almost nothing.

But notifications don't spread evenly. If one of our clients schedules a campaign at 9am they expect most of it to be delivered at that time instead of dragged out through the day. Delivering 1M inside 5 minutes works out to over 3,000 per second, and that's a single campaign.

So we'll design for surges of **5,000 notifications per second** or roughly 50x our sustained rate. This is big enough that warning lights should be going off in your head about how we handle it, that will focus our design.

Core Requirements

1. The system should guarantee at-least-once delivery with best-effort deduplication.
Meaning we'd rather send a notification twice accidentally than never at all.
2. The system should handle surges of ~5,000 notifications per second.
3. The system should deliver high priority notifications like OTPs and security alerts within 5 seconds of acceptance, even during a surge.

Below the line (out of scope):

- Strict ordering of notifications across channels.
- Compliance workflows around consent and spam regulation.
- Monitoring, alerting, and CI/CD.

These requirements aren't really independent, the best-effort deduplication makes the surges *harder* to handle. We'll still need to work through them one-by-one but it can be useful to point

out facts like this to your interviewer.



These non-functional requirements describe two very different traffic classes.

High priority notifications (OTPs, security alerts) are low volume but latency sensitive. If you're trying to log in to your account and you get the code to log in 20 minutes later you probably just assumed the website didn't work. On the other hand, the standard notifications (promos, digests) arrive in huge bursts, but nobody cares if they land ten minutes late.

Getting both through one system is the heart of this problem. It's also why our API is going to ask callers to declare a **priority** on every send.

Here's how it might look on your whiteboard:

Functional Requirements:

1. Send a notification to a user either immediately or scheduled
2. Send campaigns, immediately or scheduled.
3. Users should be able to set notification preferences

Non-functional Requirements

1. At-least-once delivery with best-effort deduplication
2. Handle surges of ~5,000 notifications per second
3. Deliver critical notifications within 5 seconds, even during a surge.

Requirements

The Set Up

Defining the Core Entities

I like to begin with a broad overview of the primary entities. At this stage we don't need every column and field. Those get fleshed out later, once the design takes shape. To satisfy our functional requirements, we'll need the following:

1. **Notification:** A single message to a single user on a single channel, along with its current status. For example, a marketing email to John at 4pm.
2. **Campaign:** A scheduled send to a group of users. It captures things like the delivery time, which channel (SMS or email), and an audience.
3. **Segment:** A named audience like "all users in Canada". Membership is maintained by whatever tool defines the audience, and we read it when a campaign fans out.
4. **User:** The recipient, along with their notification preferences and contact info.

Pay attention to the relationship between Campaign and Notification. A notification is like an *instance* of a class.



This definition-vs-instances split shows up all over system design. You'll see the same shape in the [Job Scheduler](#) breakdown (a recurring job vs its individual executions) and in calendar systems (an event vs its occurrences).

In the actual interview, this can be as simple as a short list like this. Just make sure you talk through the entities with your interviewer to ensure you are on the same page.

Entities:

- Notification
- Campaign
- Template
- Segment
- User

Entities

API or System Interface

Our first functional requirement is sending a notification to a single user. The body carries the recipient, the channel, the priority, the content, and an optional `scheduledAt` that defaults to now.

```
POST /notifications -> 200 OK { id, status }
Body: {
  userId,
  channel,    // push | email | sms
  priority,   // high | standard
  content,    // { title, body }
  scheduledAt // optional, defaults to now
}
```



We `POST` because we're creating a new resource. We're starting here by assuming that we directly call the provider when we accept the notification, which means we'll be able to return a

200 success or failure. Once delivery moves off the request path, accepting a notification and delivering it stop being the same event. But more on that later.

Campaigns get an endpoint of their own. You could stretch `POST /notifications` to accept a `segmentId` and call it a day, but then one endpoint sometimes creates a notification and sometimes creates a campaign which is unnecessary overloading.

```
POST /campaigns -> 202 Accepted { id }  
Body: {  
  segmentId,  
  channel,    // push | email | sms  
  priority,   // high | standard  
  templateId,  
  scheduledAt // optional, defaults to now  
}
```



Unlike a single send, a campaign answers with a 202 right away. The sending happens later, so there's no outcome it could report at creation time.



I haven't found many interviewers who actually care about the difference between a 200 and a 202, but it's a good thing to know about in case you are the one who finds them.

Both of these bodies grow an idempotency key before we're done. It's fine to start with the simple contract and evolve it, as long as you tell your interviewer that's what you're doing.

Users also need a say in what reaches them, which is what notification preferences are for. Preferences are a full replace of a known resource, so a PUT works nicely here and gives us an idempotent endpoint for free.

```
PUT /users/{userId}/preferences -> Preferences  
Body: {  
  optOuts,    // e.g. ["sms"]  
  quietHours  // e.g. { start: "22:00", end: "08:00", tz }  
}
```



If you've read our other breakdowns, seeing `userId` in the path might set off alarms. User identity normally comes from the `JWT` and never from the client, and passing it in the path is usually a red flag.

Here the `userId` belongs in the path, because our callers aren't end users at all. They are internal services authenticated with API keys, acting on behalf of the users whose preferences they are changing. When someone flips a toggle on the product's settings page, the product's backend calls us with a `userId` it has already authenticated.



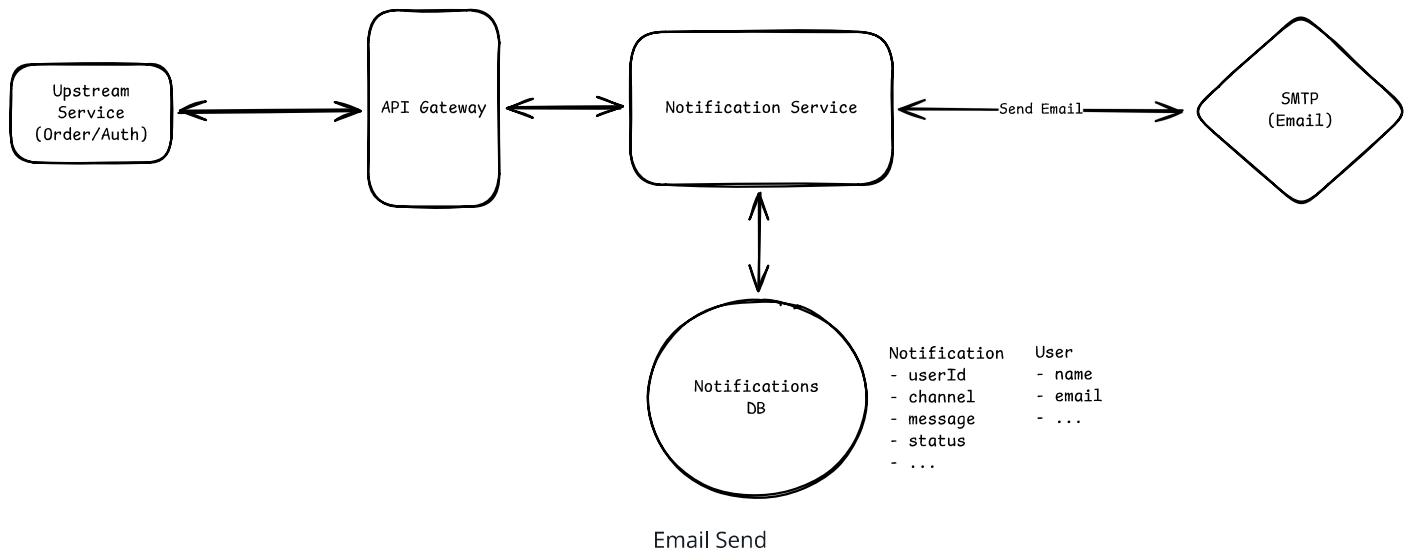
If you're interviewing with a platform team or internal tools team, they may want to know how authorization carries through the system. Often this takes the form of *delegation tokens* or *service accounts* that allow a service to act on behalf of another service or user.

High-Level Design

Next, we'll go one-by-one through the functional requirements, building the simplest thing that satisfies each one. We outlined some really juicy non-functional requirements but I would advise you to resist the urge to solve them now. It's more important we get a functional system working before we overwhelm ourselves jumping straight to the finish line.

1) Upstream services should be able to send a notification to a user, immediately or scheduled

Let's build the direct path first, a single notification sent right now, and let's make it an email. SMS follows nearly the same flow and push adds one piece on top of it, so one channel is enough to start.



1. **API Gateway:** The entry point for upstream services. The gateway authenticates each service's API key and applies rate limits so a runaway service doesn't break high-priority use-cases like authentication.
2. **Notification Service:** The core of the system. It looks up the recipient's email address, persists the notification, and calls the provider to deliver it.
3. **Database:** Stores Notification rows plus our slice of user data, like each user's email address, which syncs over from the product's user service. We'll use Postgres, though frankly any modern database works here. Nothing about a notification row needs anything crazy.
4. **Email Provider:** The service that actually gets the message into the inbox. We don't own the last mile of delivery, the provider does. Our job is to hand it well-formed requests over SMTP or its API.

When an upstream service sends a notification right now:

1. The service POSTs to `/notifications` with a `userId`, channel, and content.
2. The API Gateway authenticates the service's API key and forwards the request to the Notification Service.
3. The Notification Service looks up the user's email address from our slice of user data.
4. The service writes a Notification row with status PENDING, then calls the provider.
5. The service updates the row to SENT or FAILED based on the provider's response, and returns a 200 with the notification id and that status.

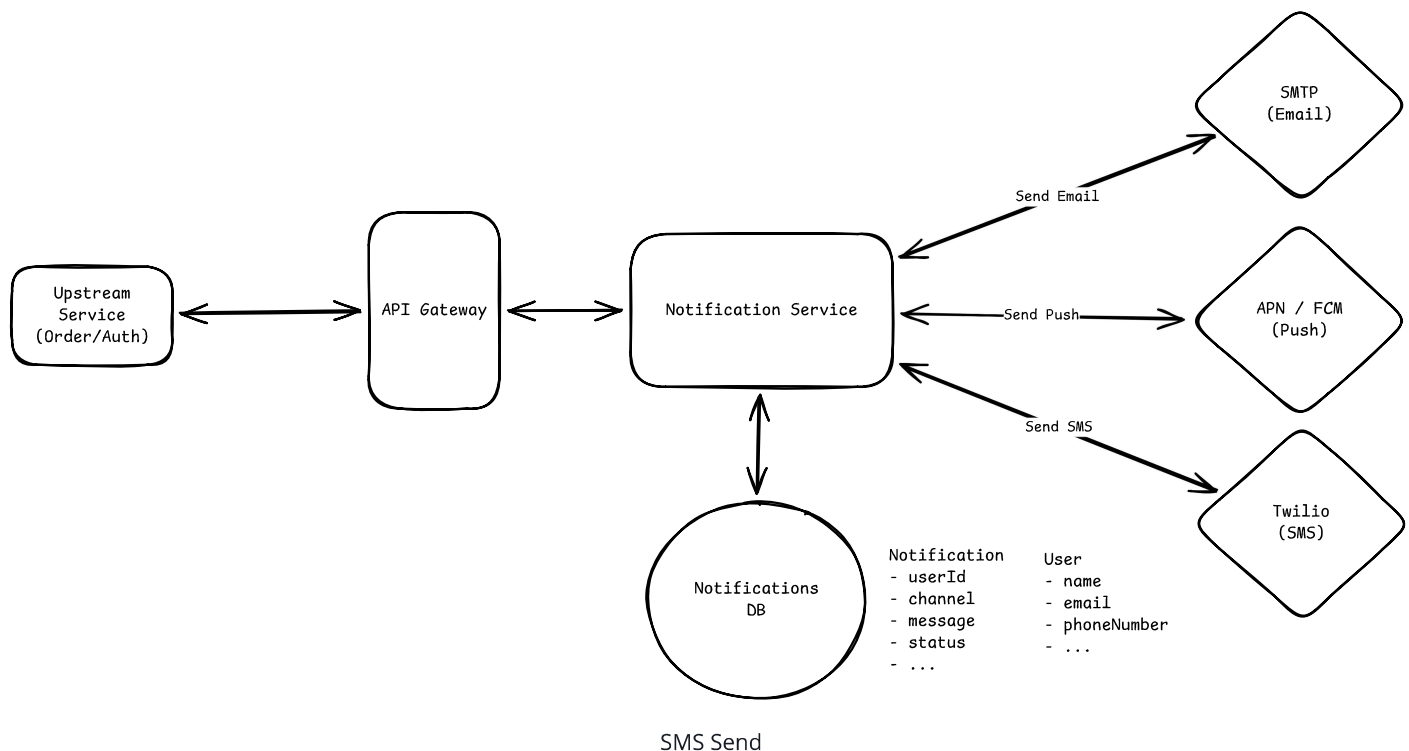
With this, a caller can send an email or SMS since these are known addresses.

Push is the only channel that needs something more from us. To send one, we hand a token for the user's device to Apple's or Google's push service, APNs or FCM. Those tokens rotate, so there's nothing stable for the user service to sync over the way it syncs email addresses and phone numbers.

Instead the product's mobile app sends us its device token directly, on login and again whenever it changes. We save it alongside the rest of our user data.

```
PUT /users/{userId}/devices -> Device
Body: {
  platform,    // ios | android
  pushToken
}
```

Once the token sits with the rest of our user data, a push send can work just like an email. The service just looks up a token instead of an address.



The most common mistake I see on this question is conflating the notification platform with the delivery provider.

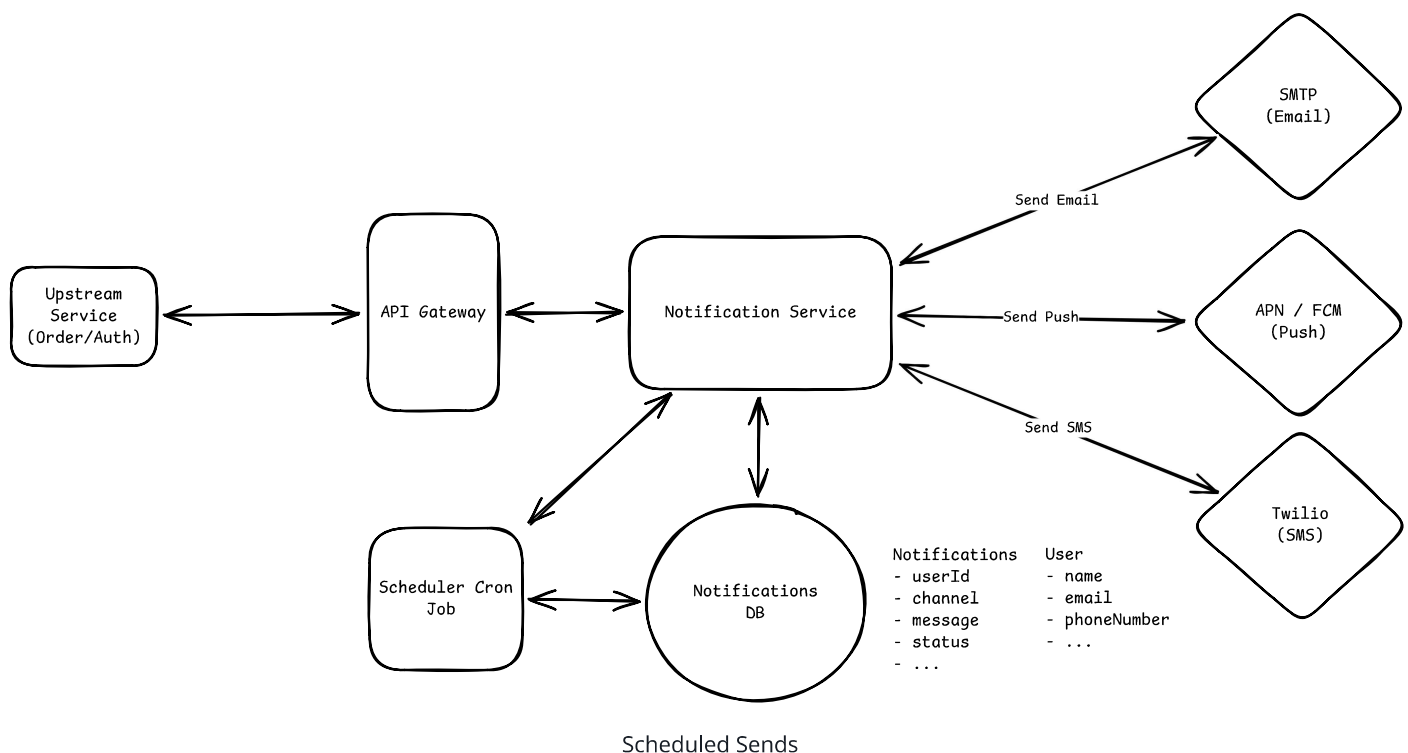
Your system is a router and a bookkeeper. It decides what to send where, and it records what happened. The carrier relationships and the device connections belong to APNs, Twilio, and the email provider.

When a candidate starts designing persistent socket connections to phones, they're solving a different and much harder problem, and a good interviewer will redirect them (unless that's what they were after!). This is where it helps to be explicit in the functional requirements.

So now we can send an email, a text, or a push notification the moment an upstream service asks for one. But what about a notification that's meant to go out later?

To schedule a send, the Notification Service writes the row with status SCHEDULED and stops there instead of calling a provider. Then we add one new component, a **scheduling cron job** that scans the Notifications table every minute for scheduled rows whose time has arrived and pushes each one through the same send path a send-now request takes.

This is fine to get started with and minute granularity is acceptable here. Anything that cares about seconds, like an OTP, comes in as a synchronous request and never touches the cron job at all.



2) Upstream services should be able to send campaigns to segments of users

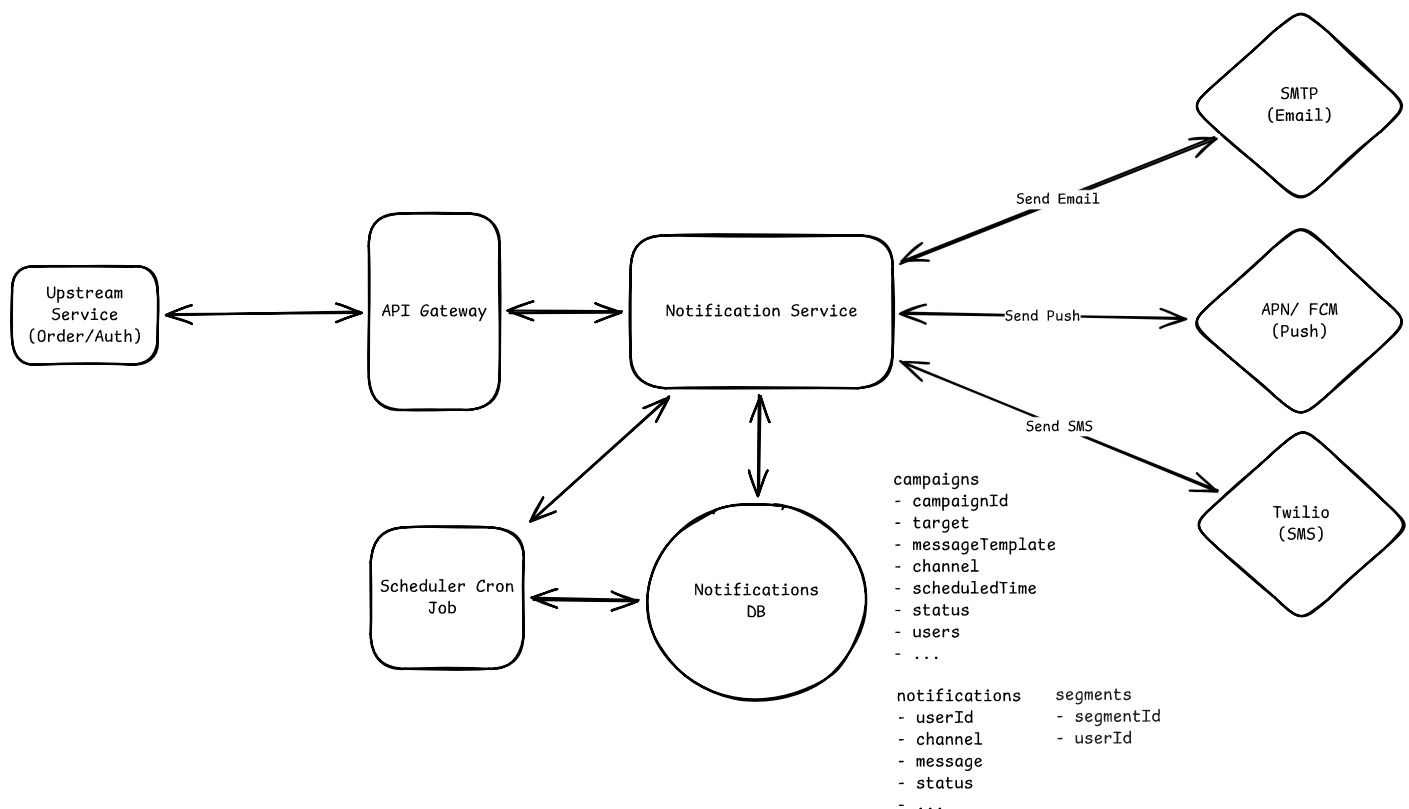
Now that we can get a notification to a single user, let's expand the design to campaigns, where one request from marketing turns into notifications for a whole segment of users.

When a caller POSTs to `/campaigns`, we write a Campaign row and hand back a 202 right away. It's just CRUD (create, read, update, delete) until we turn that one row into deliveries.

We already have a scheduling job that is polling for delayed sends, so let's expand it to point at the Campaigns table too.

1. The cron job wakes up and finds a Campaign whose `scheduledAt` has passed.
2. It reads the segment's membership to resolve the list of recipients.
3. For each recipient, it pushes a send through the same path a direct notification takes.
4. It marks the campaign complete.

We'll read the segment as a snapshot when fan-out starts. This means that someone who joins mid-publish misses this campaign and catches the next one. I think this is fine for a marketing campaign, but when I ask this I want to hear you acknowledge it (a big thing separating senior candidates from juniors is just noticing issues like this).



Campaign Sends

Now we've rolled everything together and while one pipeline for everything keeps the system simple, it creates three problems:

1. **The provider call is directly in the request path.** If Twilio has a slow day, our callers are going to have to wait on it. Worse, if our service crashes after writing the row but before calling the provider, that notification is simply gone. So much for at-least-once.
2. **A big campaign is going to fight with everything else for resources.** When the cron job expands a 1M-user campaign, all million sends might sit before the OTP code we want to give a user.
3. **A single host is making a million provider calls.** The whole fan-out is one loop on one machine, so every send in the campaign depends on that machine staying up. If it dies halfway through we have no idea where it stopped.

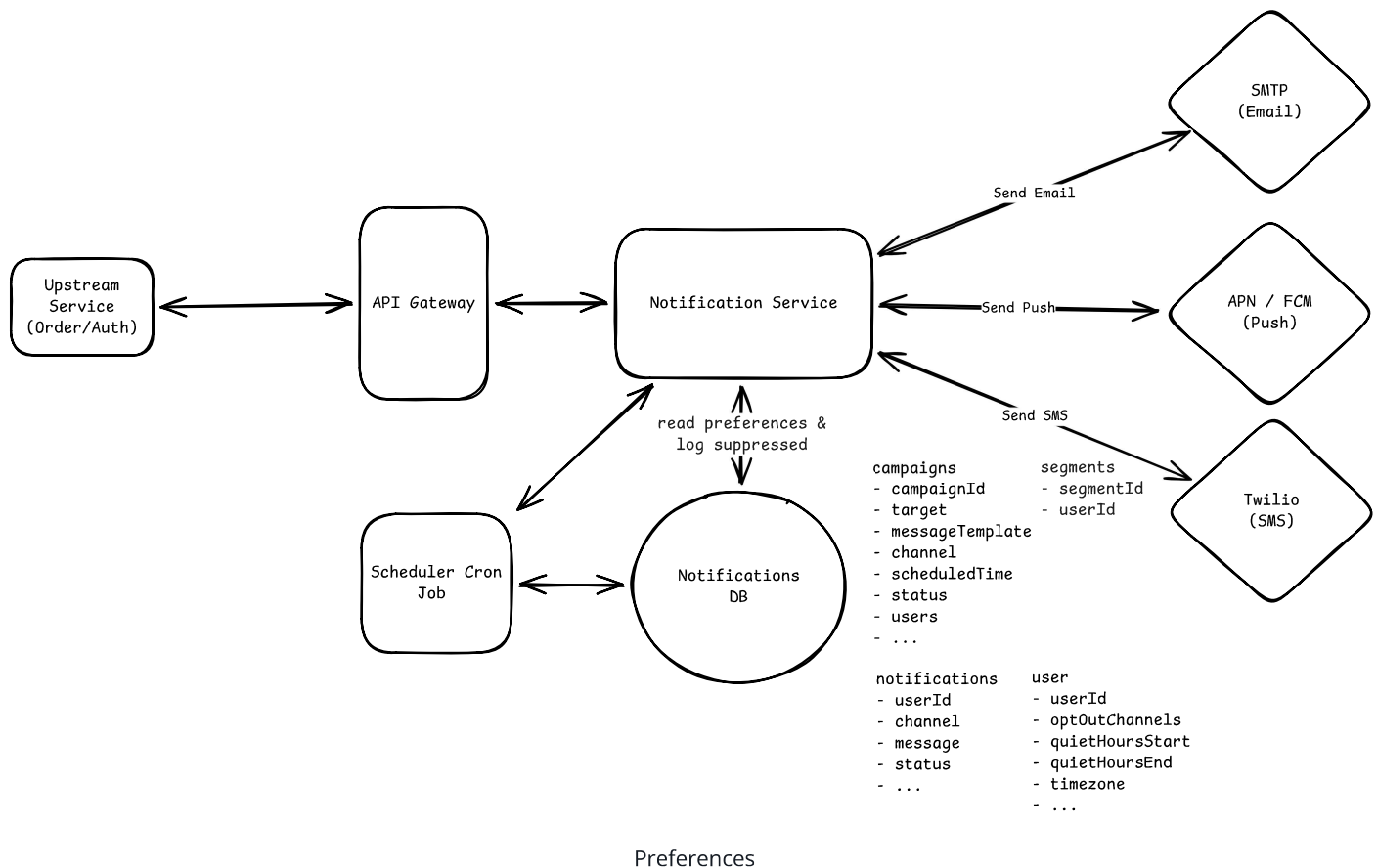
We'll come back to these, but first let's round out the functional requirements.

3) Users should be able to set notification preferences

The good news is this requirement needs no new components at all. Preferences are attributes on the User, which gets written through the preferences endpoint and routed by the Gateway to the Notification Service.

Users get an opt-out per channel and a quiet hours window. Quiet hours apply to standard notifications but not to high priority ones like OTPs and fraud alerts, of course.

Every notification gets checked against those preferences right before the provider call. When the check suppresses something, we record it as SUPPRESSED along with the reason rather than dropping it silently (this will come in handy later when we're oncall going "why the heck did this user not get this notification?!").



And with that, we have a working system. A single OTP goes out in one synchronous hop, a million-user campaign fans out through the cron job, and users have a say in what reaches them.

It works, but it doesn't satisfy a single one of our non-functional requirements. That's what the deep dives are for.

Potential Deep Dives

With a working system on the board, the non-functional requirements are what's left.

1) How do we guarantee an accepted notification is never dropped?

We flagged this problem while drawing the campaign flow. Right now the Notification Service writes the row, calls the provider inline, updates the status, and only then returns to the caller. We talked about why this sucks just a moment ago.

So we need to get delivery off the request path, and the question is what we put it on.

Pattern: Multi-step Processes

Notification delivery is a multi-step process (accept, enqueue, deliver, record) where every step can fail independently. While we won't go so far as to use something like Temporal for this problem (most interviewers will wave you off because it obscures some of the most interesting aspects of this problem), we're effectively solving the same workflow problems here.

[Learn This Pattern](#)

Bad Solution: Retry Inside the Request

Approach

The smallest possible change is to keep the synchronous call and wrap it in retries. If the provider fails, back off and try again a few times before giving up, and only then return to the caller.

It's the first thing most candidates reach for, and it does fix the narrowest version of the problem. A single flaky provider response no longer drops a notification.

Challenges

Everything else about it is worse than what we started with.

The retry loop lives in process memory, so a crash or a routine deploy still loses everything. Also, during a real provider outage, the caller's connection is held open for the length of our backoff, and the auth service starts timing out on logins because our SMS provider is degraded. The principle we aspire to with good system design is to fail fast and fail loud which this does neither.

And the caller can't tell the difference between "we gave up" and "we never got it." What we have built here is at-most-once delivery, the exact opposite of what we promised the caller.

Good Solution: Durable Records with a Polling Dispatcher



Approach

The next thing we can try is to decouple sending the notification from acknowledging the caller. All three failures came from doing both in the same request, so we'll acknowledge as soon as the notification is safely recorded and deliver it separately kinda like we do for campaigns.

The Notification Service writes the row with status PENDING and responds to the caller. A background service polls the Notifications table for PENDING rows, calls the provider for each, and marks them SENT or FAILED. Rows that fail get picked up again on a later poll. Since the row is written before anything else happens, a dispatcher that crashes loses nothing, because the work is still sitting in the table when it comes back.

The response now goes out before anyone has called a provider, so it can't report an outcome. That's the 202 we promised at the API. Callers who need to know how a send went ask for it separately.

```
POST /notifications      -> 202 Accepted { id }  
GET  /notifications/{id} -> { id, status, updatedAt }
```



Challenges

This solves for restarts, but we've just signed ourselves up to build a queue on top of Postgres, and Postgres makes a mediocre queue (hot take, there are people who love Postgres even more than we do).

One dispatcher also won't be enough, so we'll need several running in parallel. That creates a new problem: two dispatchers can pick up the same PENDING row and send the notification twice. Each dispatcher has to claim rows as it reads them, typically with `SELECT ... FOR UPDATE SKIP LOCKED` in Postgres.

We'll also have to build retries ourselves by tracking the attempt count and the next time each row should be tried. Over time, the table becomes both our queue and our history,

so every poll has to work around a growing pile of sent and failed rows unless we prune or partition it.

Lastly, the poll interval means delivery latency can be no faster than our poll interval. If we're polling every 10 seconds, a message that arrived at the beginning will have to wait 10s to do anything (which feels sluggish at best).

Great Solution: Publish First to a Durable Queue

Approach

Every awkward thing about that last approach came from making Postgres act like a queue. So let's stop building a bad queue and use a real one.

Instead of writing that PENDING row and delivering from it, the Notification Service hands the notification to a durable queue and returns its 202 once the queue has acknowledged it. The accept path no longer writes to the database, just to the queue. We'll use SQS here, though any durable message queue will do.

So the accept path is now just a single write to the queue. If we crash before the enqueue the caller gets an error and sends it again.

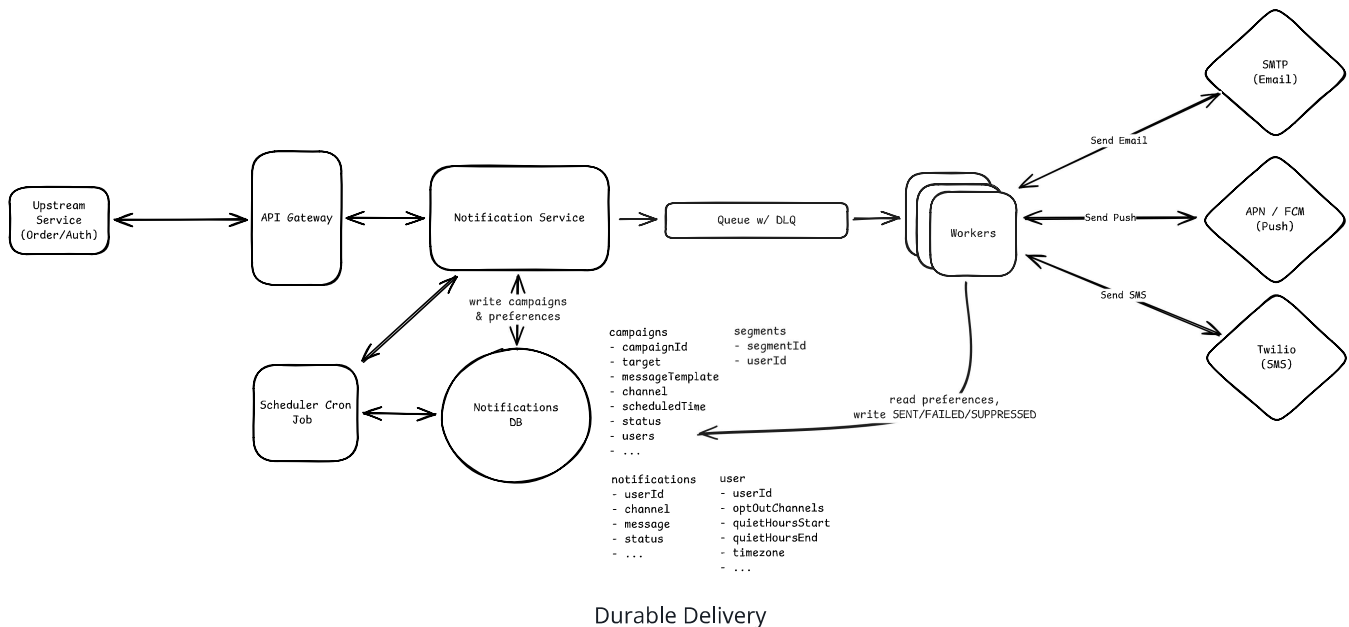
Delivery works like this:

1. A worker receives a message from the queue, which hides it from other workers for a visibility timeout.
2. It looks up the recipient's email address or device token and checks their preferences.
3. It calls the provider.
4. It writes the outcome to a Notification row in Postgres, SENT if the provider accepted it and FAILED if it didn't. A FAILED row records the attempt and leaves the message eligible for retry.
5. Only then does it delete the message from the queue.

If a worker dies before it finishes, the message stays in the queue. Once its visibility timeout expires, another worker picks it up and tries again. Provider failures work the

same way, with retries spaced out using exponential backoff. After too many failures, the message moves to a dead-letter queue for someone to inspect instead of being retried forever.

The queue handles most of this for us. We configure the retry policy, visibility timeout, and dead-letter queue rather than building that machinery ourselves.



Challenges

The system is now generating more duplication than we started with. SQS is at-least-once so can internally produce duplication even if we didn't start with a duplicate message. Beyond that, callers may have created duplication themselves, so we'll need a way to catch them.

Consuming is also destructive. Once a worker deletes a message we forget about it. If we ship a bad worker build we can't rewind this afternoon's traffic to replay it.



I've seen a lot of candidates propose writing the notification to the database first and publishing to the queue second. The challenge there is that you're introducing a dual write problem, where a crash between those two steps leaves a row that no message represents.

You can fix that with the outbox pattern, where the queue is derived from committed rows instead of published alongside them but I still don't think it's the best idea. An outbox is good when we need to commit database state and produce a message *together*. Accepting a notification is just a single publish and nothing else.

Before we move on, we picked SQS, and plenty of strong candidates pick Kafka or another durable queue. It works. The strongest case for Kafka is that being able to rewind and replay (because it's a stream) means that if you ship a bad worker build, you can reset the offset and reprocess this morning's sends instead of losing them.

This is a real advantage, but it's not something our requirements asked for, so we stayed with SQS.

Pattern: Scaling Writes

A campaign blast is a write surge, with a huge burst in throughput in a short time. Buffering it in a durable queue is one of the key strategies for scaling writes and we cover the full playbook in the pattern.

[Learn This Pattern](#)

2) How do we deliver high priority notifications within 5 seconds, even during a campaign blast?

This is the crux of the interview and where I spend the most time when I ask it. Everything up to now treated all notifications the same way, and this is where we start to talk about why that doesn't work.

Say a marketing campaign sends a notification to a million users at 9am. We can add all million notifications to the queue almost immediately, but our providers only let us send 5,000 per second across the three channels. This means backlog, but it's not an emergency just yet. The queue takes a little over three minutes to clear, which is perfectly fine for a promotion. No one expects every user to receive it at exactly 9am.

But then the auth service accepts a login OTP seconds later. It goes into the same queue the promos are in, a million messages back, and at 5,000 sends per second it waits **200 seconds**. Now we have the problem.

Bad Solution: Throw More Workers at It



Approach

This is the first answer I get maybe half the time. The queue is backed up, so add capacity to drain it faster. Autoscale the worker pool on how far behind the queue is, throw money and AWS "elastic" resources at it, and get to that OTP sooner.

Challenges

If we could scale to 100x in an instant, this might work. But bringing up servers typically takes seconds to minutes and realistically this type of scaling is only possible with a lot of infrastructure and a lot of money.

In order to drain 1M messages in under 5 seconds, we would require 200,000 sends per second, which is 40x our design target. AWS puts in place account limits on how many servers you can spin up in a moment to prevent workloads like this. It's great for you if you can get it, but it's horrifically costly for them. *Somebody* has to keep all those idle servers when you're not using them, so it depletes their reserves (and their profitability).

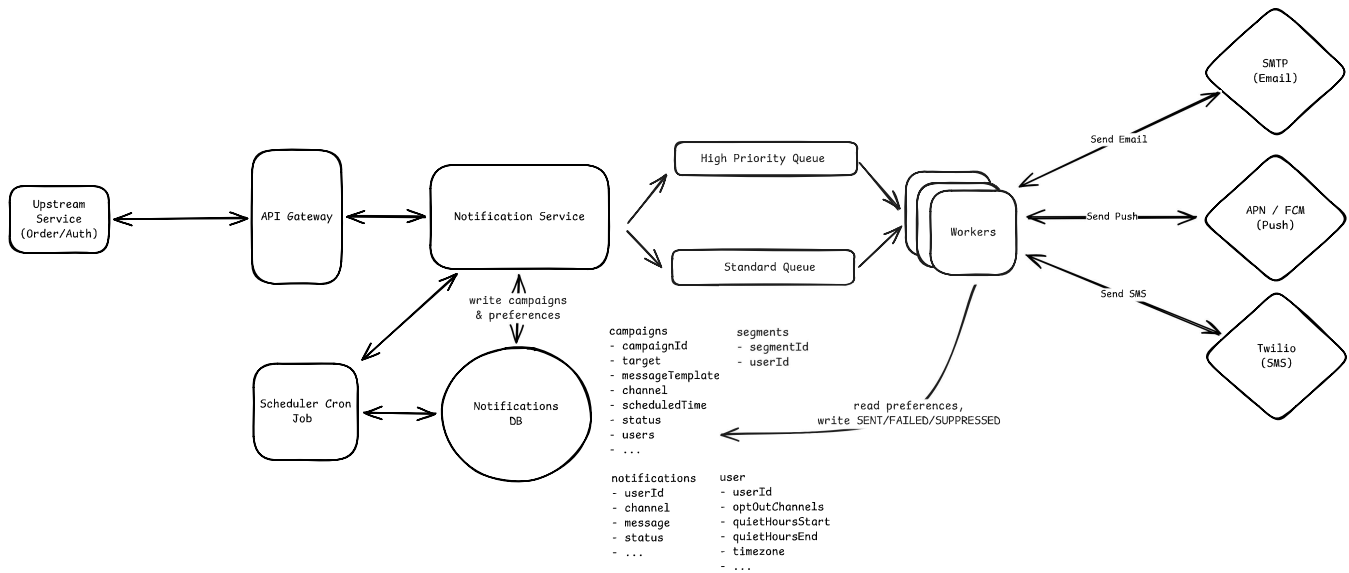
Good Solution: Separate Queues per Priority Tier



Approach

A significant improvement is to split the queue by the priority the caller declared at the API, giving us a high priority queue and a standard queue. We'll have the service send notifications to the appropriate queue, and we'll set up the workers to drain the high priority queue first.

This means our OTP sits in a small queue that gets served first even while our marketing queue is completely backed up.



Separate Queues

Challenges

We've solved queue position, but everything downstream of it is still shared.

Think about our providers. If standard traffic is consuming our entire Twilio rate allocation, the high priority SMS is going to get throttled by them even if it sits in a small queue on our side.

We have a similar version of the same issue in our compute. Workers that drain the high priority queue first do serve it quickly in the happy case, but we need to have the standard pipeline behave itself. One bad outcome would be that the worker itself is stuck behind a slow provider call and thus holding up the high priority queue. Ew.

Great Solution: Independent Worker Pools with Per-Tier Provider Budgets



Approach

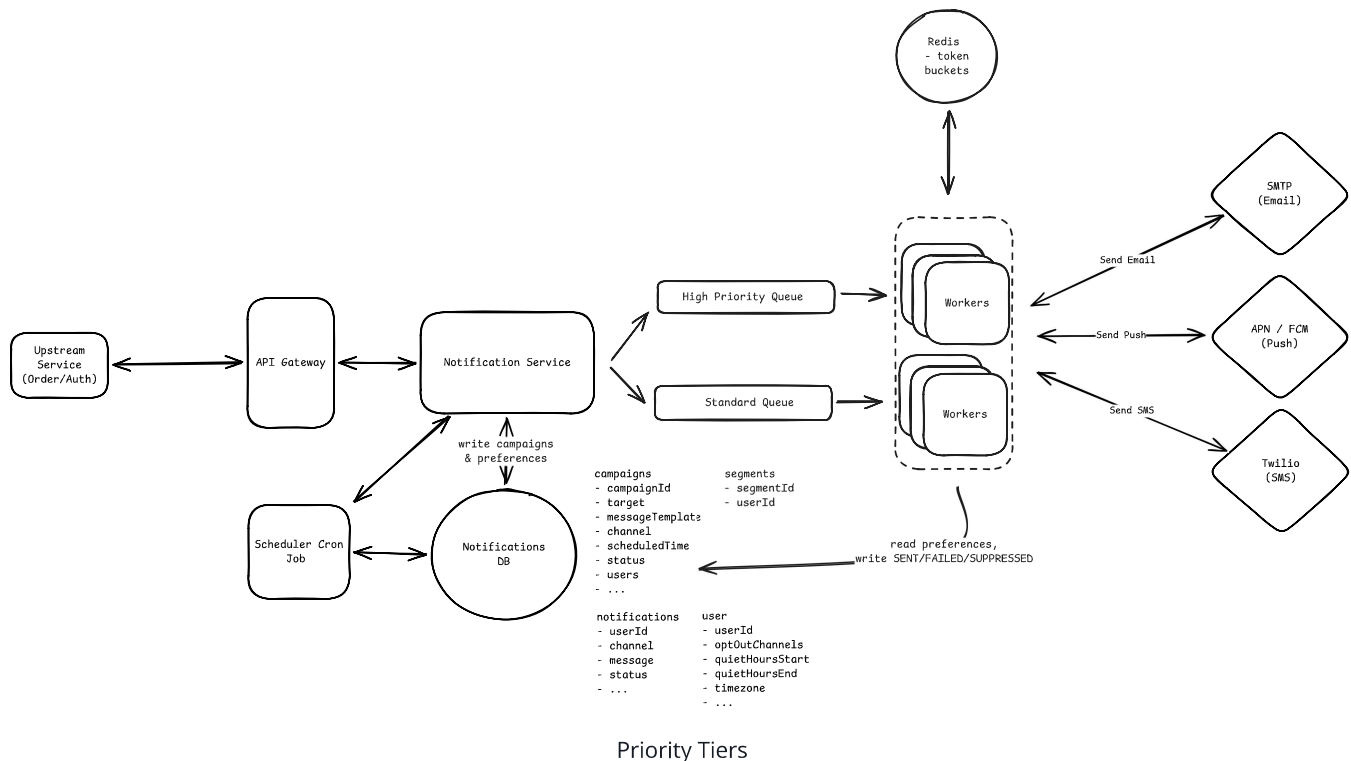
You're probably seeing the writing on the wall here. We need more isolation (separating queues and worker pools) and more control (provider budgets and token buckets).

Say Twilio sells us 1,000 SMS per second. Today any tier can spend all 1,000 of them leading to our OTP being starved.

So we can carve that allocation up by tier and enforce it with a shared token bucket in Redis. Every worker draws a token before it calls the provider, and we refill the bucket at the top of each one-second window under two rules:

1. High priority gets a reserved slice of the bucket. Say, 100 of the 1,000 requests per second are allocated just for the high priority group.
2. Standard gets the remaining 900 and can't touch the reserved 100, no matter how far behind it falls.

We should give the high priority tier its own compute too. Its queue gets a separate pool of workers.



Challenges

The high priority pool sits underused most of the time, but that's a deliberate choice worth the tradeoff. Keeping the pool ready ensures it's available for OTPs and fraud alerts.

The budget adds a Redis check in front of every provider call, which is one more moving part to build and get right. But again, that's what we want.

Surviving a provider brownout

A slow provider is the other way to blow our 5 second SLA. Some of this is unavoidable, we can't violate physics to get our notifications delivered faster. But there are some optimizations we can make and they are great things to offer in the interview (or be prepared to explain if they decide to ask you about them).

Say Twilio's API p99 climbs from under a second to 30 seconds. Our workers don't know that, so they keep calling, and each stuck call pins a worker for 30 seconds. While a provider is browned out we cannot deliver everything on time, so the job is to keep high priority messages moving and let everything else wait it out.

There's 3 typical ways to handle this:

1. **Circuit breaker.** We track the failure and timeout rate per provider. Once it crosses a threshold the breaker opens, so calls fail immediately instead of waiting out a 30-second timeout, and we shed non-essential load to leave room for high priority messages.
2. **Bulkhead.** Cap how many calls to one provider can be in flight at once like a semaphore. When Twilio has its allotment outstanding, no more workers start a call to it, so we're not spinning all our capacity on a service that's mostly down.
3. **Provider router.** Give the channels that have substitutes a primary and a backup. We can get clever with this in the UI too, if SMS is down maybe email is the only option we offer for authentication.

3) How do we prevent duplicate notifications on top of at-least-once delivery?

Up until this point, every decision we've made has correctly favored at-least-once delivery. The downside is that sending duplicate messages is now a lot more likely.

For example, when we pull a notification off the queue, a worker can call Twilio and then die before it deletes the message. The SMS *was* actually sent, but the queue doesn't know this. So once the visibility timeout in SQS runs out, another worker pulls that same message and sends the notification again.

Callers can create duplicates too. You could have an upstream service that POSTs a new notification, times out, and then POSTs it again. You could also have our cron job crash halfway

through fanning out a segment, and when it runs again everything that was already added goes back into the queue.

True exactly-once delivery is impossible to achieve, but we can go a long way toward keeping duplicates rare.

Good Solution: Claim the ID Before Sending



Great Solution: Check Before the Send, Record After the Ack

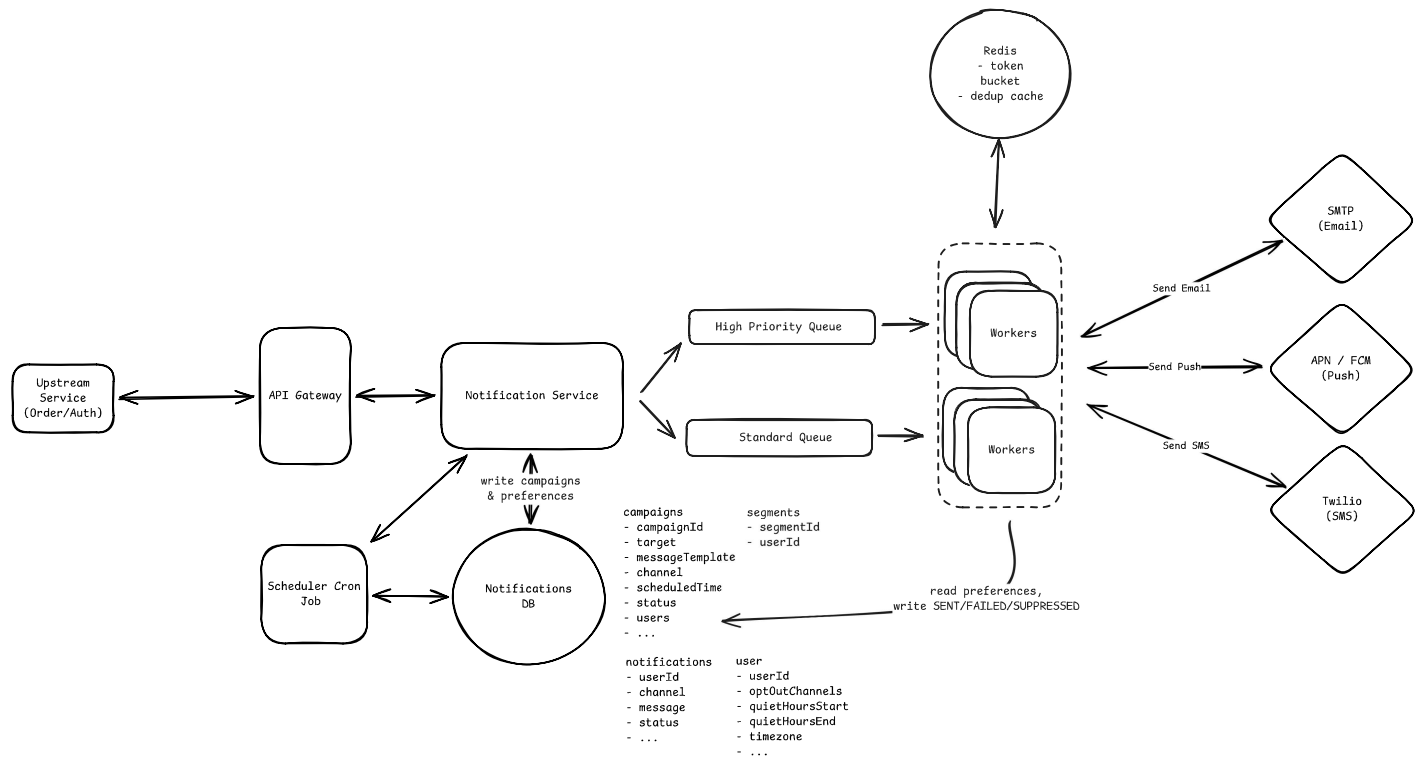




Name your delivery semantics explicitly in the interview. Exactly-once stops at the provider boundary, so the honest design is at-least-once with deduplication at each point duplicates can enter. You should be able to point at each of those points on your diagram.

Final Design

Putting it all together, here's where we land:



Final Design

Some additional deep dives you might consider

An interviewer can take this problem plenty of places we didn't go:

- 1. Send-time optimization.** We delivered marketing's 9am campaign at 9am. The fancier version asks when each user actually engages and picks a per-user send time inside a window marketing allows. Our pipeline already handles millions of small scheduled sends, so the interesting part is interacting with an offline ML model that scores those send times. If your interviewer has an ML background, they'll probably want to talk about this.
- 2. Frequency capping.** A real platform eventually caps how many notifications a user gets per day across all types. A good start is next to the preference check at dispatch time, but we need to talk about counters.
- 3. Campaign cancellation.** Marketing schedules a blast, then legal wants it stopped. Or we spotted a typo. We expect this to be rare, but serious. How can we check for cancellations without dragging on performance?

4. **Timezone-aware delivery.** "9am" usually means 9am for the user, which means that campaign is probably spread out over 24 hours. The scheduling machinery we already built fits, just with two dozen due times instead of one.

What is Expected at Each Level?

We've built more here than any single 45-minute interview could cover. What's actually expected depends on your level.

Mid-level

For this question, an E4 candidate will have clearly defined the API and the data model. That means send and campaign endpoints each carrying a priority, and a Campaign that fans out into Notifications.

From there I want a working high-level design covering both the send-now path and scheduled campaigns. I'm looking for them to recognize that the synchronous call to a provider can drop notifications, and with some prompting to land at least the good solution for durable delivery. Priority isolation and the dedup ordering aren't things I would expect them to land without some help from me.

Senior

E5 candidates should speed through the initial high-level design so we can spend our time in the deep dives, driving both durable delivery and high priority tier isolation without me steering.

There's a lot of finer details about isolation, queues, and prioritization that we touched on in this breakdown that I would want the candidate to be able to confidently discuss. The more they're thinking ahead of potential problems the better. This problem is surprisingly complex and deep, and a good hallmark of a senior candidate is they're able to follow that depth without me holding their hand.

A senior candidate probably won't reach per-tier provider budgets or the dedupe ordering on their own, and that's fine.

Staff+

For a staff+ candidate, expectations are high on depth across all three deep dives.

Unprompted, they put numbers on the problem to see where the issues lie. They also know the ceiling is the providers rather than our own workers, which is the thing that stops us from just adding capacity at every step.

I'm also looking for judgment in the failure modes. What should the system do when the dedup store is down? When is failover to a backup provider worth its cost? (High priority tier only.)

Why does claiming an idempotency key before the provider call turn at-least-once into at-most-once?

The best staff candidates teach me something, usually an operational constraint that never shows up on a diagram.

Test Your Knowledge

Answer the question below to find your gaps.

Question 1 of 15

Which channel needs contact details the mobile app sends us directly rather than data synced over from the product's user service?

- 1 SMS, since carrier routing details change per region
- 2 All three, since our slice of user data is written only by the client apps
- 3 Push, because device tokens rotate and there's nothing stable to sync
- 4 Email, because address changes have to be verified before we can use them